Week 9 Technical Report - Approximation Algorithms

Project repository (complete source, tests, benchmarks and results):
https://github.com/RDeibel2025/CTX-Advanced-Algorithms
Approximation algorithms: src/approximation/ · demonstration: examples/week9_demo.py
Benchmark: benchmarks/week9_approx_benchmark.py

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026

📎 Complete project repository

https://github.com/RDeibel2025/CTX-Advanced-Algorithms

1. Executive Summary

I implemented vertex cover by maximal matching and by LP rounding, set cover by greedy and by deterministic and randomized LP rounding, greedy facility location, MST-doubling metric TSP, and randomized and local-search Max-Cut, and benchmarked them against exact optima. Every guarantee here is a comparison with a lower bound that can be computed in polynomial time, and the report reads each result through that bound. Dividing cost by the bound certified the vertex cover 2-approximation at 1.705 times optimal at n = 2,000, where the exact optimum is out of reach. Where an optimum existed, the measured ratios were 1.531 for vertex cover, 1.098 for greedy set cover and 1.189 for MST doubling, all below their proven bounds.

2. Approximation Theory Overview

For a minimization problem an algorithm has ratio ρ if its cost is at most ρ times the optimum OPT; for a maximization problem, if its value is at least OPT / ρ (here reported as value / OPT, at least 1/2 for Max-Cut). OPT is the quantity no polynomial algorithm is known to compute, so every proof replaces it with a bound L that is computable and never better than OPT: if cost ≤ ρL and L ≤ OPT, then cost ≤ ρ OPT (CLRS Ch. 35; Amakobe, 2025, Ch. 9). This project uses four such bounds. A maximal matching M bounds vertex cover, since every cover needs an endpoint of each matched edge. The LP relaxation bounds vertex cover and set cover, since relaxing integrality can only lower the minimum. The minimum spanning tree bounds metric TSP, since deleting one edge of the optimal tour leaves a spanning tree. The edge count |E| bounds Max-Cut from above.

A PTAS achieves ratio 1 + ε for every fixed ε > 0 in time polynomial in n, possibly exponential in 1/ε; an FPTAS is polynomial in both, as subset sum and knapsack are after scaling and rounding the values (Vazirani, 2001). APX-hard problems, such as vertex cover, metric TSP and Max-Cut, have no PTAS unless P = NP. Some problems allow no constant ratio at all: general TSP without the triangle inequality has none unless P = NP (CLRS Ch. 35), and set cover cannot beat a logarithmic ratio by much, which is why Chvátal's (1979) greedy H(n) bound is the standard (Vazirani, 2001).

3. Methodology

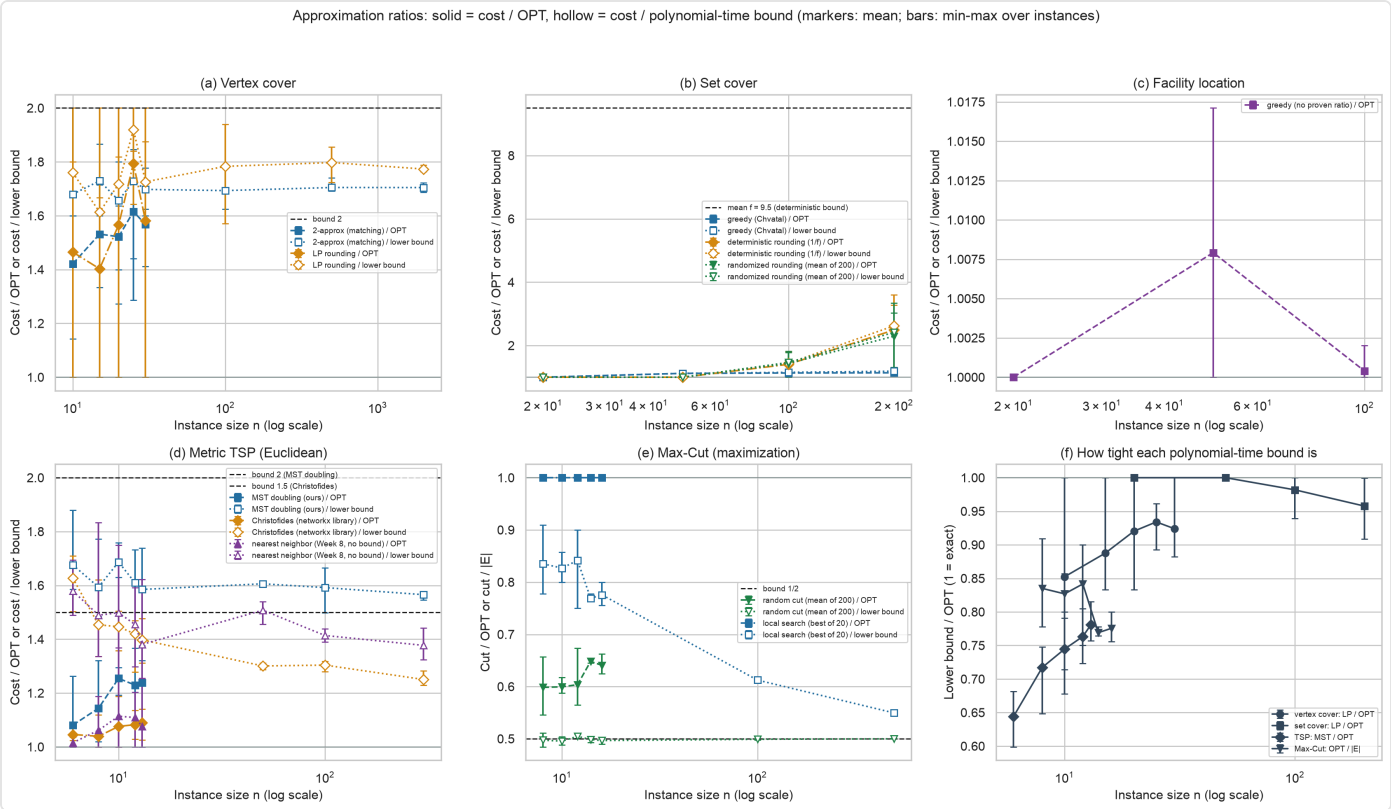
Machine and solver. Apple M2 Max, 64 GB RAM, macOS 14.5 (arm64), CPython 3.12.4. Every LP and MILP is solved by SciPy's HiGHS through `linprog` and `milp`. Times use `time.perf_counter` with garbage collection paused; a call under 20 ms is retimed as the mean of 5 runs. The full benchmark took 79.7 s.

Instances. Vertex cover: 5 G(n, p) graphs per n with average degree 4, exact optimum (Week 8's branching solver) for n = 10 to 30 and approximations to n = 2,000. Set cover: 5 instances per n = 20, 50, 100 and 200 elements with n/2 sets, density 0.1 and costs in [1, 10]. Facility location: 5 instances each of 20, 50 and 100 clients with 10, 15 and 20 facilities, Euclidean distances and opening costs in [20, 60]. Metric TSP: Euclidean points, 5 instances per n = 6 to 13 with the Held-Karp optimum from Week 6 and 3 per n = 50, 100 and 300; every instance passed `is_metric` before MST doubling ran. Max-Cut: 3 G(n, 0.3) graphs per n, brute force to n = 16 and 100 and 500 without it. Randomized rounding ran 200 times per instance, the random cut 200 times per graph and local search from 20 starts. No size had to be cut.

Seeds. The master seed is 42. Every instance and every random run comes from `derive_seed(42, experiment, n, rep[, run])`; the instance file records 130 distinct instance seeds, and the study made 8,558 method runs.

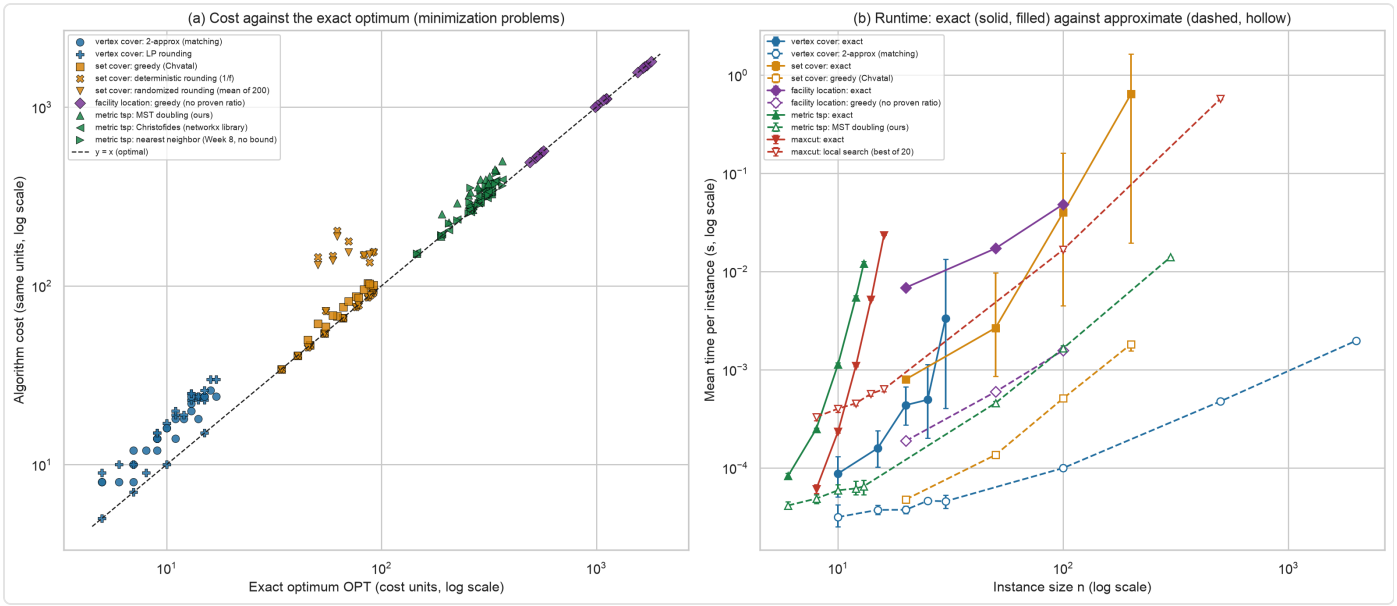
Metrics. Ratio to OPT where the optimum exists, and ratio to the lower bound at every size (cut / |E| for Max-Cut). Christofides comes from networkx and is a labelled library comparison, not my code. The repository holds every week's project, so Week 9's tests and outputs sit in their own `tests/week9/` and `benchmarks/results/week9/` folders under the required names.

4. Results



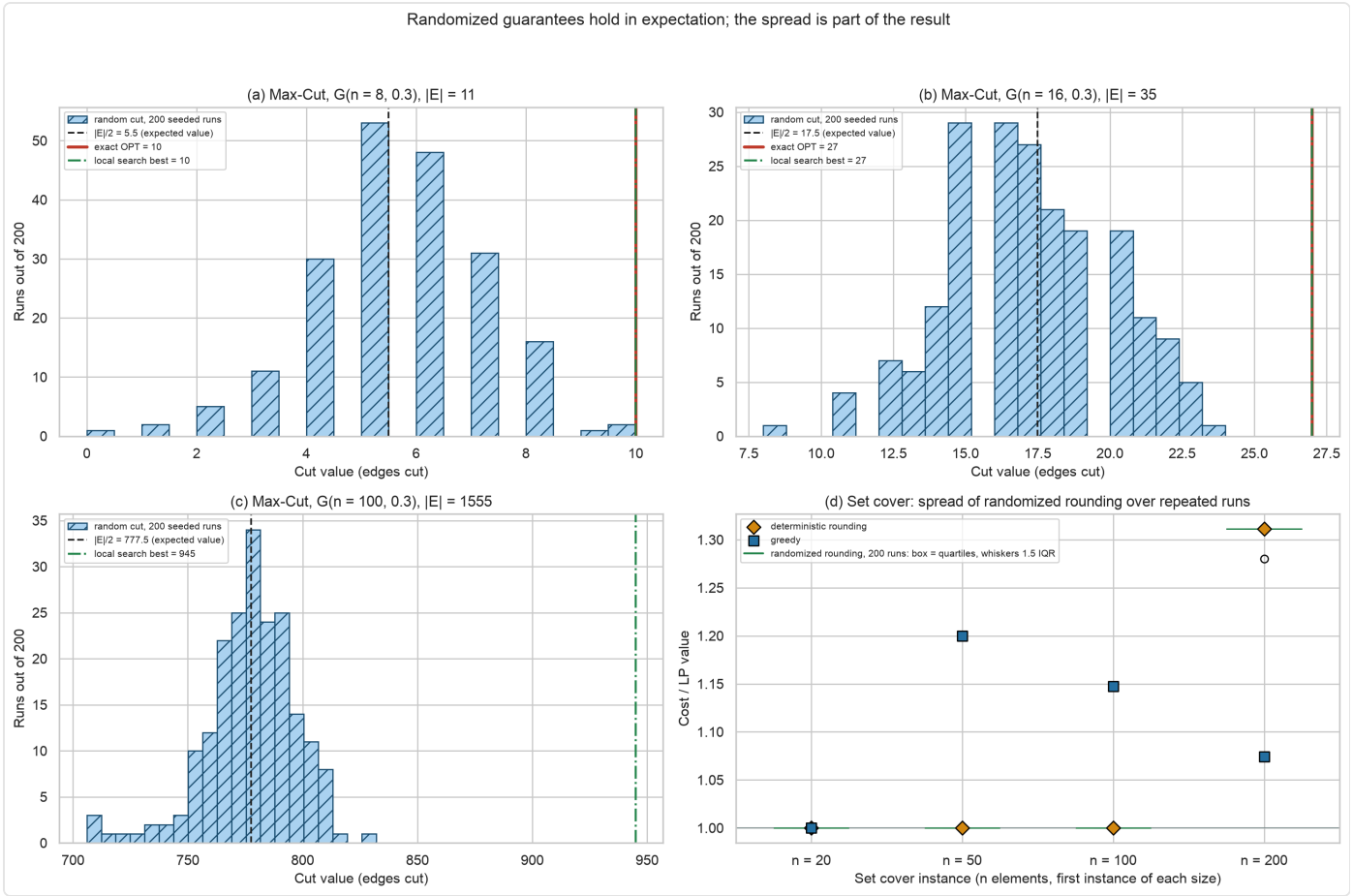
Vertex cover. Over 25 graphs with an optimum, the 2-approximation averaged 1.531 (1.143 to 1.846) and was never optimal; LP rounding averaged 1.562 (1.000 to 1.923) and was optimal on 4. The LP value was 0.852 to 0.934 of OPT and the matching 0.710 to 0.808, so the LP is the tighter bound. At n = 2,000 the LP value was 979.7 against a matching of 835, which certifies the 2-approximation's 1,670-vertex cover at 1.705 times optimal at most. The 2-approximation took 2.0 ms there and LP rounding 207 ms.

Set cover. Greedy averaged 1.098 times OPT over 20 instances (at most 1.216), far inside its H(d) bound, which averaged 3.99 at 200 elements. Rounding depends on the LP. At 20 and 50 elements the LP was integral, and both roundings were optimal with zero variance. At 200 elements the LP was 0.958 of OPT, but each element lay in up to 20 sets, and deterministic rounding cost 2.491 times OPT and randomized rounding 2.294. Randomized rounding's within-instance standard deviation was 8.12 there (coefficient of variation 0.053), and its worst single run was 3.482 times OPT. No run needed a fix-up.



Facility location. Greedy averaged 1.0028 times the MILP optimum (at most 1.0171) and was optimal on 10 of 15 instances. It has no proven ratio and no lower bound, so outside these instances nothing certifies it.

Metric TSP. The MST was 0.730 of the optimal tour (0.599 to 0.815). MST doubling averaged 1.189 times OPT (at most 1.375), Christofides 1.067 and nearest neighbor 1.075. At n = 300, divided by the MST, the three certify at 1.565, 1.250 and 1.377; Christofides took 0.522 s against MST doubling's 0.014 s.



Max-Cut. The random cut's mean was 0.9972 of |E|/2, as the expectation argument predicts, and 0.618 of OPT on the small graphs. Single runs varied widely: the worst cut no edges at all. Local search, best of 20 starts, found the optimum on all 15 small graphs. At n = 500 it cut 0.550 of |E| against the random cut's 0.500, in 0.574 s for all 20 starts.

5. Discussion

Approximation shines where the bound is close to OPT and cheap. The vertex cover LP averaged 0.852 to 0.934 of the optimum at the sizes where both exist, so dividing by it gave a certificate of 1.705 at n = 2,000 in one LP solve. Exact optimization was feasible here only because the sizes were small: the exact set cover MILP took 0.640 s at 200 elements, and Week 8's exact vertex cover solver already needed 2.67 s at n = 60.

The accuracy measurements and the guarantees disagree in a useful way. MST doubling has a proven factor of 2 and was the worst TSP method on average, while nearest neighbor, with no bound, did better. Greedy facility location was within 2% of optimal with no bound at all. A bound is a promise about the worst case; it does not rank methods on typical instances. A heuristic's good average only holds for the instances tested, while a bound holds for every input.

Randomization changes what the guarantee says. Max-Cut's |E|/2 and randomized rounding's O(log n) hold in expectation, and the measured spread shows a single run can be much worse. Repetition and local search turn that expectation into a reliable result at a small cost.

The trade-off between accuracy and runtime is steep in places. Christofides cut the certified TSP ratio from 1.565 to 1.250 at 37.2 times the time. At n = 2,000, LP rounding cost 105 times the 2-approximation's time and produced a slightly larger cover; its value is the LP bound it computes.

6. Conclusion

In practice, approximation algorithms are useful less for their worst-case constant than for the lower bound that comes with them. That bound turns an unknown optimum into a measurable gap on every instance. At n = 2,000, a 2-millisecond matching and one LP solve showed that a vertex cover was within 1.705 of optimal, a statement no exact solver could have checked at that size.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Ch. 35 Approximation Algorithms.
- Amakobe, M. (2025). *Advanced Computational Algorithms*, Ch. 9 (open textbook).
- Christofides, N. (1976). *Worst-case analysis of a new heuristic for the travelling salesman problem* (Report 388). Carnegie-Mellon University.
- Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research, 4*(3), 233-235.
- Vazirani, V. V. (2001). *Approximation Algorithms*. Springer.

AI use

Claude (Claude Code) drafted the code, tests and this report's first draft from a specification I wrote from the instructions and required reading, including the hand-checked values the tests assert. Every measurement comes from running the benchmark on my machine, and every number in this report is printed from the CSVs by `tools/week9_facts.py`. Full disclosure: `docs/AI_USE.md`.